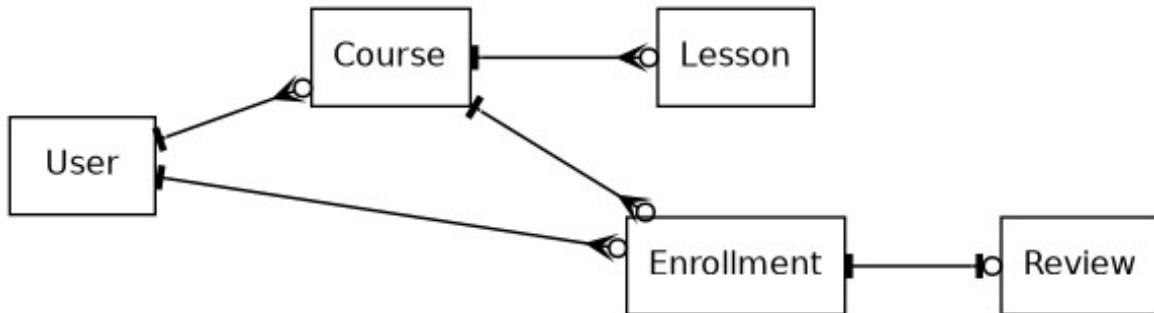


ДЗ №1. Онлайн-курсы

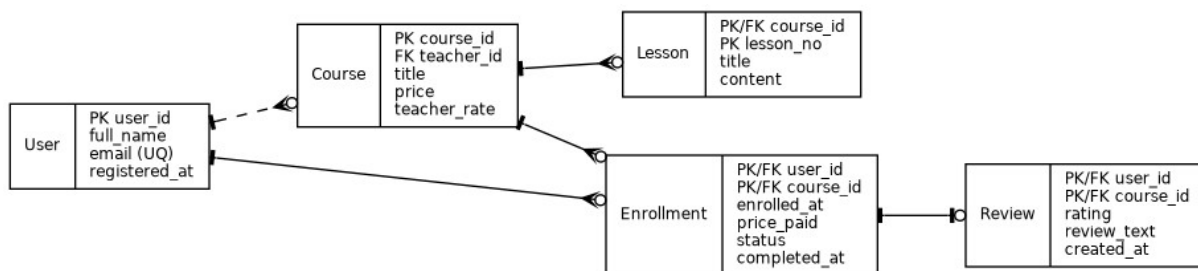
1. Концептуальная модель

Основные сущности: User, Course, Lesson, Enrollment, Review.



User может быть преподавателем курса и одновременно студентом. Курс состоит из уроков. Enrollment хранит запись студента на курс. Review относится к этой записи.

2. Логическая модель



Обозначения: PK - первичный ключ, FK - внешний ключ, UQ - уникальное значение. В диаграмме используется нотация Crow's Foot.

User -> Course - связь 1:N и неидентифицирующая, потому что teacher_id не входит в PK Course. Связи Course -> Lesson, User/Course -> Enrollment и Enrollment -> Review - идентифицирующие: ключ родителя входит в ключ дочерней таблицы.

Как требования учтены в модели

- Почта пользователя обязательна и уникальна.
- У курса есть преподаватель, название, цена и доля преподавателя.
- Уроки нумеруются внутри курса. Поэтому у Lesson составной ключ (course_id, lesson_no).
- Enrollment связывает студентов и курсы. Один студент не может два раза записаться на один курс.
- Оценка в Review ограничена значениями от 1 до 5.
- Статус обучения хранится в Enrollment.

3. Физическая модель

PostgreSQL. Скрипт пересоздаёт только учебную схему hw_edtech.

```
BEGIN;
```

```
DROP SCHEMA IF EXISTS hw_edtech CASCADE;
```

```
CREATE SCHEMA hw_edtech;
```

```
SET search_path TO hw_edtech;
```

```
CREATE TABLE users (  
    user_id SERIAL PRIMARY KEY,  
    full_name VARCHAR(150) NOT NULL,  
    email VARCHAR(254) NOT NULL UNIQUE,  
    registered_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE courses (  
    course_id SERIAL PRIMARY KEY,  
    teacher_id INTEGER NOT NULL REFERENCES users(user_id),  
    title VARCHAR(200) NOT NULL,  
    price DECIMAL(10,2) NOT NULL CHECK (price >= 0),  
    teacher_rate DECIMAL(4,2) NOT NULL CHECK (teacher_rate BETWEEN 0 AND 1)  
);
```

```
CREATE INDEX idx_courses_teacher  
ON courses(teacher_id);
```

```
CREATE TABLE lessons (  
    course_id INTEGER NOT NULL REFERENCES courses(course_id),  
    lesson_no INTEGER NOT NULL CHECK (lesson_no > 0),  
    title VARCHAR(200) NOT NULL,  
    content TEXT,  
    PRIMARY KEY (course_id, lesson_no)  
);
```

```
CREATE TABLE enrollments (  
    user_id INTEGER NOT NULL REFERENCES users(user_id),  
    course_id INTEGER NOT NULL REFERENCES courses(course_id),  
    enrolled_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    price_paid DECIMAL(10,2) NOT NULL CHECK (price_paid >= 0),  
    status VARCHAR(20) NOT NULL DEFAULT 'active'  
    CHECK (status IN ('active', 'completed')),  
    completed_at TIMESTAMP,  
    PRIMARY KEY (user_id, course_id)  
);
```

```
CREATE INDEX idx_enrollments_course  
ON enrollments(course_id);
```

```
CREATE TABLE reviews (  
    user_id INTEGER NOT NULL,  
    course_id INTEGER NOT NULL,
```

```
rating INTEGER NOT NULL CONSTRAINT reviews_rating_check
    CHECK (rating BETWEEN 1 AND 5),
review_text TEXT,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (user_id, course_id),
FOREIGN KEY (user_id, course_id)
    REFERENCES enrollments(user_id, course_id)
);

CREATE INDEX idx_reviews_course
ON reviews(course_id);

COMMIT;
```

4. Частые запросы

1. Показать курсы конкретного преподавателя и количество студентов на каждом курсе.
2. Показать уроки выбранного курса по порядку.
3. Показать курсы конкретного студента и статус обучения.
4. Показать отзывы выбранного курса.
5. Посчитать начисления преподавателю за месяц по его курсам.